

第四章:

p122 4.3 实现京东首页的轮播广告效果，至少包括 3 个产品广告的图片。

```
public class SlideShowView extends FrameLayout {
    private ImageLoader imageLoader = ImageLoader.getInstance();

    //轮播图图片数量
    private final static int IMAGE_COUNT = 5;
    //自动轮播的时间间隔
    private final static int TIME_INTERVAL = 5;
    //自动轮播启用开关
    private final static boolean isAutoPlay = true;

    //自定义轮播图的资源
    private String[] imageUrls;
    //放轮播图片的 ImageView 的 list
    private List<ImageView> imageViewsList;
    //放圆点的 View 的 list
    private List<View> dotViewsList;

    private ViewPager viewPager;
    //当前轮播页
    private int currentItem = 0;
    //定时任务
    private ScheduledExecutorService scheduledExecutorService;

    private Context context;

    //Handler
    private Handler handler = new Handler(){

        @Override
        public void handleMessage(Message msg) {
            // TODO Auto-generated method stub
            super.handleMessage(msg);
            viewPager.setCurrentItem(currentItem);
        }

    };

    public SlideShowView(Context context) {
        this(context,null);
        // TODO Auto-generated constructor stub
    }

    public SlideShowView(Context context, AttributeSet attrs) {
```

```

        this(context, attrs, 0);
        // TODO Auto-generated constructor stub
    }

    public SlideShowView(Context context, AttributeSet attrs, int defStyle) {
        super(context, attrs, defStyle);
        this.context = context;

        initImageLoader(context);

        initData();
        if(isAutoPlay){
            startPlay();
        }

    }

    /**
     * 开始轮播图切换
     */
    private void startPlay(){
        scheduledExecutorService = Executors.newSingleThreadScheduledExecutor();
        scheduledExecutorService.scheduleAtFixedRate(new SlideShowTask(), 1, 4,
TimeUnit.SECONDS);
    }

    /**
     * 停止轮播图切换
     */
    private void stopPlay(){
        scheduledExecutorService.shutdown();
    }

    /**
     * 初始化相关 Data
     */
    private void initData(){
        imageViewsList = new ArrayList<ImageView>();
        dotViewsList = new ArrayList<View>();

        // 一步任务获取图片
        new GetListTask().execute("");
    }

    /**
     * 初始化 Views 等 UI
     */
    private void initUI(Context context){
        if(imageUrls == null || imageUrls.length == 0)

```

```

        return;

LayoutInflater.from(context).inflate(R.layout.layout_slideshow, this, true);

LinearLayout dotLayout = (LinearLayout)findViewById(R.id.dotLayout);
dotLayout.removeAllViews();

// 热点个数与图片特殊相等
for (int i = 0; i < imageUrls.length; i++) {
    ImageView view = new ImageView(context);
    view.setTag(imageUrls[i]);
    if(i==0)//给一个默认图
        view.setBackgroundResource(R.drawable.appmain_subject_1);
    view.setScaleType(ScaleType.FIT_XY);
    imageViewsList.add(view);

    ImageView dotView = new ImageView(context);
    LinearLayout.LayoutParams params = new
LinearLayout.LayoutParams(LayoutParams.WRAP_CONTENT,LayoutParams.WRAP_CONTENT);

    params.leftMargin = 4;
    params.rightMargin = 4;
    dotLayout.addView(dotView, params);
    dotViewsList.add(dotView);
}

viewPager = (ViewPager) findViewById(R.id.viewPager);
viewPager.setFocusable(true);

viewPager.setAdapter(new MyPagerAdapter());
viewPager.setOnPageChangeListener(new MyPageChangeListener());
}

/**
 * 填充 ViewPager 的页面适配器
 */
private class MyPagerAdapter extends PagerAdapter{

    @Override
    public void destroyItem(View container, int position, Object object) {
        // TODO Auto-generated method stub
        (((ViewPager)container).removeView((View)object);
        ((ViewPager)container).removeView(imageViewsList.get(position));
    }
}

```

```
}
```

```
@Override
```

```
public Object instantiateItem(View container, int position) {
```

```
    ImageView imageView = imageViewsList.get(position);
```

```
    imageLoader.displayImage(imageView.getTag() + "", imageView);
```

```
    ((ViewPager)container).addView(imageViewsList.get(position));
```

```
    return imageViewsList.get(position);
```

```
}
```

```
@Override
```

```
public int getCount() {
```

```
    // TODO Auto-generated method stub
```

```
    return imageViewsList.size();
```

```
}
```

```
@Override
```

```
public boolean isViewFromObject(View arg0, Object arg1) {
```

```
    // TODO Auto-generated method stub
```

```
    return arg0 == arg1;
```

```
}
```

```
@Override
```

```
public void restoreState(Parcelable arg0, ClassLoader arg1) {
```

```
    // TODO Auto-generated method stub
```

```
}
```

```
@Override
```

```
public Parcelable saveState() {
```

```
    // TODO Auto-generated method stub
```

```
    return null;
```

```
}
```

```
@Override
```

```
public void startUpdate(View arg0) {
```

```
    // TODO Auto-generated method stub
```

```
}
```

```
@Override
```

```
public void finishUpdate(View arg0) {
```

```
    // TODO Auto-generated method stub
```

```

    }

}

/**
 * ViewPager 的监听器
 * 当 ViewPager 中页面的状态发生改变时调用
 *
 */
private class MyPageChangeListener implements OnPageChangeListener{

    boolean isAutoPlay = false;

    @Override
    public void onPageScrollStateChanged(int arg0) {
        // TODO Auto-generated method stub
        switch (arg0) {
            case 1:// 手势滑动, 空闲中
                isAutoPlay = false;
                break;
            case 2:// 界面切换中
                isAutoPlay = true;
                break;
            case 0:// 滑动结束, 即切换完毕或者加载完毕
                // 当前为最后一张, 此时从右向左滑, 则切换到第一张
                if (viewPager.getCurrentItem() == viewPager.getAdapter().getCount() - 1
&& !isAutoPlay) {
                    viewPager.setCurrentItem(0);
                }
                // 当前为第一张, 此时从左向右滑, 则切换到最后一张
                else if (viewPager.getCurrentItem() == 0 && !isAutoPlay) {
                    viewPager.setCurrentItem(viewPager.getAdapter().getCount() - 1);
                }
                break;
        }
    }

    @Override
    public void onPageScrolled(int arg0, float arg1, int arg2) {
        // TODO Auto-generated method stub

    }

    @Override

```

```

        public void onPageSelected(int pos) {
            // TODO Auto-generated method stub

            currentItem = pos;
            for(int i=0;i < dotViewsList.size();i++){
                if(i == pos){

                    ((View)dotViewsList.get(pos)).setBackgroundResource(R.drawable.dot_focus);
                }else {

                    ((View)dotViewsList.get(i)).setBackgroundResource(R.drawable.dot_blur);
                }
            }
        }

        /**
         *执行轮播图切换任务
         */
        private class SlideShowTask implements Runnable{

            @Override
            public void run() {
                // TODO Auto-generated method stub
                synchronized (viewPager) {
                    currentItem = (currentItem+1)%imageViewsList.size();
                    handler.obtainMessage().sendToTarget();
                }
            }
        }

        /**
         * 销毁 ImageView 资源，回收内存
         */
        private void destoryBitmaps() {

            for (int i = 0; i < IMAGE_COUNT; i++) {
                ImageView imageView = imageViewsList.get(i);
                Drawable drawable = imageView.getDrawable();
                if (drawable != null) {

```

```

        //解除 drawable 对 view 的引用
        drawable.setCallback(null);
    }
}

/**
 * 异步任务,获取数据
 */
class GetListTask extends AsyncTask<String, Integer, Boolean> {

    @Override
    protected Boolean doInBackground(String... params) {
        try {
            // 这里一般调用服务端接口获取一组轮播图片，下面是从百度找的几个
            图片

            imageUrl = new String[]{
                "http://image.zcool.com.cn/56/35/1303967876491.jpg",
                "http://image.zcool.com.cn/59/54/m_1303967870670.jpg",
                "http://image.zcool.com.cn/47/19/1280115949992.jpg",
                "http://image.zcool.com.cn/59/11/m_1303967844788.jpg"
            };
            return true;
        } catch (Exception e) {
            e.printStackTrace();
            return false;
        }
    }

    @Override
    protected void onPostExecute(Boolean result) {
        super.onPostExecute(result);
        if (result) {
            initUI(context);
        }
    }
}

/**
 * ImageLoader 图片组件初始化
 */

```

```

    * @param context
    */
    public static void initImageLoader(Context context) {
        // This configuration tuning is custom. You can tune every option, you
        // may tune some of them,
        // or you can create default configuration by
        // ImageLoaderConfiguration.createDefault(this);
        // method.
        ImageLoaderConfiguration config = new
        ImageLoaderConfiguration.Builder(context).threadPriority(Thread.NORM_PRIORITY
        -
        2).denyCacheImageMultipleSizesInMemory().discCacheFileNameGenerator(new
        Md5FileNameGenerator()).tasksProcessingOrder(QueueProcessingType.LIFO).writeDebugLo
        gs() // Remove
        // for
        // release
        // app
        .build();
        // Initialize ImageLoader with configuration.
        ImageLoader.getInstance().init(config);
    }

```


